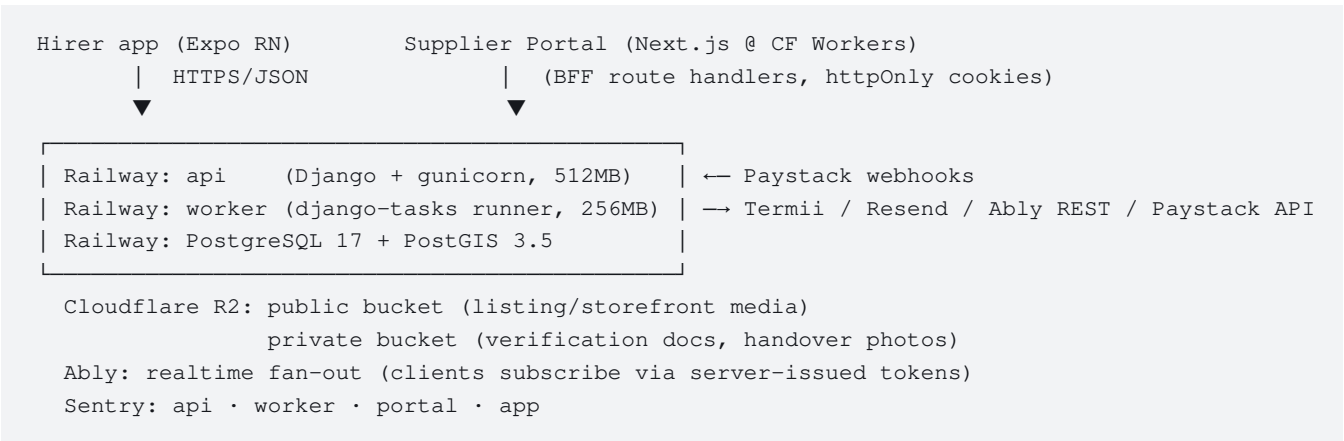


TERMINAL — Technical Specification Document v2.1

v2.1 · June 2026 · Implements FSD v2.1 under DECISIONS D-001...D-014 Status: ✓ Reference edition for development Stack (D-009...D-013): Django 6.0.x · DRF 3.17 · PostgreSQL 17 + PostGIS 3.5 · django-tasks · Ably · Railway Hobby · Cloudflare R2 · MapLibre GL / OpenFreeMap / LocationIQ · Next.js on Cloudflare Workers · React Native + Expo · Paystack · Termii · Resend · Sentry Budget ceiling: **\$25/month** total (04 \$8: fixed ≈ \$10–15).

1. System Topology



One region (Railway EU-west); Nigeria RTT ~120–180ms — acceptable at MVP. **No Redis anywhere** (D-010). The DB is the broker, the queue, and the source of truth.

2. Repository, Environments, CI/CD

2.1 Monorepo layout

```
terminal/
├── design.md                # coding-agent engineering guide (root)
├── docs/                    # all v2 planning + this TSD/FSD (+ PDFs)
├── backend/
│   ├── manage.py
│   ├── pyproject.toml      # deps via uv/pip-tools; ruff config
│   ├── settings/          base.py dev.py prod.py test.py
│   ├── core/              accounts/ suppliers/ listings/ search/
│   ├── hires/             payments/ messaging/ ops/
│   └── tests/              # cross-app integration tests
├── portal/                  # Next.js 15 App Router (@opennextjs/cloudflare)
│   ├── src/app             src/components src/lib(bff, api-client) wrangler.toml
├── app/                     # Expo RN, TypeScript, expo-router
│   ├── app/                components/ lib/ assets/
├── packages/tokens/         # design tokens JSON → tailwind.tokens.js, tokens.ts, glyphs/
├── docker-compose.yml       # postgres:17-3.5 + mailpit (dev)
└── .github/workflows/      # backend.yml portal.yml (path-filtered)
```

2.2 Environments & config

- **dev:** docker-compose (PostGIS + mailpit); Paystack/Termii/Ably test keys; `DEBUG=1`. **prod:** Railway +

Workers. **No staging in MVP** — Paystack test mode + feature flags bridge; staging returns in Phase 2.

- `django-enviro`; `.env.example` exhaustive and CI-checked. Key vars:

Var	Notes
<code>DATABASE_URL</code>	postgres:// scheme
<code>SECRET_KEY</code> , <code>ALLOWED_HOSTS</code> , <code>CORS_ALLOWED_ORIGINS</code>	prod from Railway env
<code>FIELD_ENCRYPTION_KEY</code>	Fernet key for bank numbers
<code>R2_ACCOUNT_ID</code> / <code>ACCESS_KEY_ID</code> / <code>SECRET</code> / <code>PUBLIC_BUCKET</code> / <code>PRIVATE_BUCKET</code> / <code>PUBLIC_BASE_URL</code>	
<code>PAYSTACK_SECRET_KEY</code> / <code>PUBLIC_KEY</code> / <code>WEBHOOK_SECRET</code>	webhook secret = account secret (HMAC)
<code>ABLY_API_KEY</code> · <code>TERMII_API_KEY</code> / <code>SENDER_ID</code> · <code>RESEND_API_KEY</code> · <code>LOCATIONIQ_KEY</code>	absent in dev ⇒ console/log fallback, never crash
<code>SENTRY_DSN</code> / <code>ENVIRONMENT</code>	

2.3 CI/CD

- Backend PR gate: `ruff` (lint+format) + `mypy` (loose) + `pytest` against PostGIS service container; coverage gates 85% `hires+payments`, 70% overall.
- Portal PR gate: `eslint` + `tsc` + `build`. App: `tsc` + `jest` unit (no CI device builds).
- Deploy: merge to `main` → Railway auto-deploy (`api+worker`; release phase runs `migrate`) · portal via wrangler GH action. Mobile: local Android / EAS-free iOS, manual cadence; OTA via expo-updates for JS-only changes.

3. Backend Design

3.1 Apps & ownership

App	Owns
<code>core</code>	<code>BaseModel</code> (UUIDv7 PK, created/updated), permissions (<code>IsSupplier</code> , <code>IsHirer</code> , <code>IsVerified</code>), pagination, error envelope, <code>core.money</code> , <code>healthz/readyz</code> , R2 presign service
<code>accounts</code>	<code>User</code> (custom, migration 0001), OTP, JWT, password reset, <code>VerificationRequest</code>
<code>suppliers</code>	<code>SupplierProfile</code> , <code>Yard</code> , storefront read API, strikes
<code>listings</code>	<code>Listing</code> , <code>Unit</code> , <code>SpecTemplate</code> (+ validation), photos, <code>Report</code> , duplicate action
<code>search</code>	Map + list search, yard aggregation
<code>hires</code>	<code>Hire</code> , <code>HireEvent</code> , <code>HandoverRecord</code> ; state machine <code>hires/state.py</code> ; fees <code>hires/fees.py</code> ; availability <code>hires/availability.py</code>
<code>payments</code>	<code>Payment</code> , <code>Refund</code> , <code>Payout</code> , <code>PaystackEvent</code> ; client, webhook, reconciliation
<code>messaging</code>	<code>Conversation</code> , <code>Message</code> , masking, Ably token endpoint, unread counters
<code>ops</code>	Admin site, queues, dashboard, dispute actions, weekly digest

Service-layer rule (binding): views/serializers never mutate domain state. All writes flow through `services.py` functions taking `(actor, ...)` and returning domain results. The state machine, fee engine, and availability check are pure, importable, and unit-tested in isolation. Signals are not used for business logic (only for cache-ish denorm if ever needed); side-effects dispatch via `transaction.on_commit(lambda: task.enqueue(...))`.

3.2 Money (`core.money`)

- All amounts are **integer kobo** (`BigIntegerField`) — Paystack-native, no Decimal drift. Helpers:
`kobo(naira)`, `naira(kobo)`, `display(kobo)` → "₦1,250,000". Kobo never shown in UI (whole-naira product).
- **Fee engine** (`hires/fees.py`, `pure`):

```
RATE_TABLE = { # D-002; per mille to stay integer (%)
    "plant_machinery": {"daily": 120, "weekly": 100, "monthly": 80},
    "trucks_haulage": {"daily": 110, "weekly": 90, "monthly": 70},
    "warehousing": {"daily": 100, "weekly": 80, "monthly": 60},
    "terminals_yards": {"daily": 100, "weekly": 80, "monthly": 60},
    "land_staging": {"daily": 100, "weekly": 80, "monthly": 60},
}
FEE_FLOOR = 250_000 # kobo = ₦2,500

def price_hire(listing, days) -> Price:
    candidates = {} # only schemes with a rate set
    if listing.daily_price: candidates["daily"] = listing.daily_price * days
    if listing.weekly_price: candidates["weekly"] = listing.weekly_price * ceil(days/7)
    if listing.monthly_price: candidates["monthly"] = listing.monthly_price * ceil(days/30)
    scheme = min(candidates, key=lambda s: (candidates[s], -SCHEME_LENGTH[s])) # D-008, tie→longer
    hire_value = candidates[scheme]
    rate = RATE_TABLE[listing.asset_class][scheme]
    service_fee = max(hire_value * rate // 1000, FEE_FLOOR)
    return Price(scheme, hire_value, service_fee, hire_value - service_fee,
        fee_basis=f"{rate/10:.0f}% {scheme} (min ₦2,500)")
```

FSD §3.1 worked examples are the canonical test vectors; add hypothesis property tests (monotonicity, floor, payout+fee≡value).

3.3 Schema (normative; all tables UUID PK + timestamps unless noted)

users — email citext uniq · phone uniq (E.164) · password (bcrypt 12) · is_supplier, is_hirer · account_level enum(basic, verified, business_verified) · is_active · suspended_at/reason · deleted_at (soft). **otp_codes** — user FK · code_hash · purpose · expires_at · attempts. (Rate limits enforced in service.) **supplier_profiles** — user 1:1 · business_name/description/logo_key · bank_name · bank_account_number_enc (Fernet) · bank_account_name · notif_prefs (4 × bool) · strike_count. **yards** — supplier FK · name · point geography(Point,4326) GIST · address_text · city. **spec_templates** — asset_class · asset_type · version · fields jsonb (defs per doc 05) · uniq(class, type, version). Seeded by data migration from docs/v2/05. **listings** — supplier FK · yard FK null (ON DELETE PROTECT) · asset_class enum · asset_type · title ≤120 · description · specs jsonb · spec_template_version · daily/weekly/monthly_price kobo (weekly/monthly null) · unit_count ≥1 · point (denorm from yard) GIST · address_text · city · status enum(draft, live, paused, archived, removed) · tier enum(basic, verified, inspected) · report_count · completeness_score · removed_reason. Indexes: (status, asset_class) · (supplier, status) · GIN on specs (jsonb_path_ops) for ★ filters. **listing_photos** — listing FK · r2_key · position · is_cover. ≤10 enforced in service. **units** — listing FK · label (only when supplier labels units). **hires** — listing FK PROTECT · hirer FK · supplier FK (denorm) · yard FK null (denorm) · start_date, end_date, duration_days · scheme · fee_basis · hire_value, service_fee, payout_amount (kobo, **locked at acceptance**) · status enum(requested, accepted, confirmed, on_hire, completed, declined, expired, cancelled, in_dispute) · cancelled_by enum(hirer, supplier, ops, system) null · decline_reason/cancel_reason · hirer_note · request_expires_at · payment_deadline null · acknowledgments jsonb · parent_hire null. Indexes: (**listing, status, start_date, end_date**) (availability hot path) · (hirer, status) · (supplier, status) · partial on holding states. **hire_events** — hire FK · actor_kind(user/system/ops) + actor FK null · from_status, to_status · meta jsonb · created_at. **Append-only** (DB: revoke UPDATE/DELETE from app role; admin readonly). **handover_records** — hire FK · kind(on_hire, off_hire) uniq with hire · photos jsonb (private R2 keys) · reading numeric null + reading_kind · notes · supplier_confirmed_at, hirer_confirmed_at. **payments** — hire FK · paystack_reference uniq · authorization_url · amount kobo · state(initiated, success, failed, abandoned) · paid_at · channel.

paystack_events — event_id uniq (webhook dedup) · payload jsonb · processed_at. **refunds** — hire FK · amount · state(pending, completed, failed) · paystack_ref · reason. **payouts** — hire FK uniq · supplier FK · amount · state(pending, due, paid, frozen) · paid_ref · paid_at · frozen_reason. **conversations** — kind(enquiry, hire) · listing FK null · hire FK null uniq · supplier FK, hirer FK · partial uniq (listing, hirer) where kind=enquiry and listing not null. **messages** — conversation FK · sender FK · body · body_masked null · sent_at · read_at null. Serving rule: body_masked or body until conversation unmask condition (hire paid); enquiry conversations always masked. **verification_requests** — user FK · kind(identity, business) · doc_keys jsonb (private bucket) · state(pending, approved, rejected) · reviewer FK null · reason. **reports** — listing FK · reporter FK · reason enum · state(open, dismissed, warned, removed) · resolution note.

3.4 Availability & the concurrency contract

Holding states: confirmed, on_hire, and accepted with payment_deadline > now().

```
SELECT count(*) FROM hires
WHERE listing_id = :listing AND status IN ('accepted','confirmed','on_hire')
AND (status <> 'accepted' OR payment_deadline > now())
AND start_date <= :end AND end_date >= :start;
-- range available iff count < listing.unit_count
```

Race rule (binding): accept_hire and apply_payment_success execute in a transaction that first takes SELECT ... FOR UPDATE on the listing row, then re-checks availability, then writes. Two concurrent payments on the last unit cannot both pass; the loser's payment triggers automatic refund (full) + apology notification — this path must be tested. On entering confirmed, the same transaction auto-declines overflowed requested/accepted hires and queues their notifications on commit.

3.5 State machine & timers

- hires/state.py: TRANSITIONS: {(from_status, action): to_status} + per-transition guards (actor role, payment state, acknowledgment presence). apply(actor, hire, action, **meta): validate → lock (listing FOR UPDATE where capacity-relevant) → write → append HireEvent → on-commit side-effects (email, Ably, badge counters).
- **Sweeps (django-tasks, every 5 min, idempotent):** requested past request_expires_at → expired · accepted past payment_deadline → cancelled(system, payment_expired) · confirmed past start+24h → on_hire · on_hire past end+48h undisputed → completed (+ payout due).
- **Daily:** Paystack reconciliation (transactions list vs payments; mismatch → Sentry + Ops email). **Weekly:** founder metrics digest; R2 orphan sweep.
- django-tasks worker runs as the Railway worker service; scheduled invocation via worker-side scheduler; every task idempotent (re-running a sweep is a no-op).

3.6 Paystack integration (D-006)

- **Initialize** (on accept): POST /transaction/initialize — amount=hire_value (kobo), channels=["card", "bank_transfer", "ussd"], metadata={hire_id}, reference=f"THR-{hire.short_id}-{attempt}". Store reference + authorization_url.
- **Webhook** POST /api/v1/payments/webhook/: verify x-paystack-signature = HMAC-SHA512(raw body, secret); insert event_id (duplicate ⇒ 200 skip); enqueue handler; return 200 fast. Handler calls GET /transaction/verify/:reference (belt-and-braces) before state.apply(system, hire, "pay").
- Client redirects are **never** trusted; the app polls hire status after browser return.
- **Refunds:** POST /refund with computed amount (FSD \$7.6); processor fee non-recoverable (≈N2,000 cap) — absorbed per cancellation rules.
- Paystack **Starter** business at launch (manual payouts need no Transfers); CAC registration in motion for Phase 2 rails. Collection fee: 1.5%+N100 capped N2,000 (waived <N2,500) — local channels only;

international cards rejected in MVP (3.9% uncapped breaks fee math).

3.7 Search & map aggregation

GET /api/v1/search/map params: bbox or (lat,lng,radius_km) · asset_class · q · price_min/max · spec_min/max (★ field per class) →

```
{ "yards": [ { "yard_id", "name", "point", "supplier": { "id", "name", "logo", "badge" },
               "listing_count", "matching_count", "class_mix": [ "plant_machinery", "trucks_haulage" ],
               "price_from", "listings": [ { "id", "title", "asset_class", "price_from", "photo", "availab
               "listings": [ { "id", "title", "asset_class", "point", "price_from", "distance_km", "photo", "badge" } ] }
```

- Yards aggregate **server-side** (GROUP BY yard over Live listings) — authoritative counts (Fleet A2/A4); summaries ride along so the Yard Sheet opens with zero extra round-trips (E4). `listings[]` carries solo pins only.
- Distance: `ST_Distance(point, ref::geography)/1000` rounded 0.1km; order by distance. Viewport: `point && ST_MakeEnvelope(...)` + GIST. ★ spec filters via JSONB containment/range on GIN.
- Client-side concerns: MapLibre clustering of *returned* pins (spatial), pin styling per doc 08 §4.2.

3.8 API conventions & inventory

- Base /api/v1/; OpenAPI at /api/docs/ (drf-spectacular) — **each module's schema is published before its frontend wave starts; the schema is the cross-team handshake.**
- Auth: Authorization: Bearer <access>; simplejwt rotating refresh + blacklist. Errors: { "error": { "code", "message", "fields?" } } with stable code strings (e.g. `availability_conflict`, `payment_window_expired`, `verification_required`, `basic_cap_exceeded`). Cursor pagination on lists. Throttles: anon search 60/min · auth 120/min · OTP send 3/h/phone · login 5/15min/IP · report 5/day/user.

Area	Endpoints
auth	POST register · POST otp/verify · POST otp/resend · POST login · POST token/refresh · POST logout · POST password-reset · POST password-reset/confirm
me	GET/PATCH me · POST me/activate-supplier · POST me/verification (presigned docs) · GET me/verification
suppliers	GET/PATCH suppliers/me/profile · GET/POST yards · PATCH/DELETE yards/:id · GET storefronts/:supplier_id (public)
listings	GET/POST listings (mine) · GET listings/:id (public if Live) · PATCH listings/:id · POST :id/publish
search	GET search/map · GET search/list (same params + cursor + group_by=asset location)
hires	POST hires (listing, dates, note) · GET hires?role=&status= · GET hires/:id (+events[], role-shaped financials per D-014) · POST :id/accept
payments	POST payments/webhook (Paystack only, signature-gated)
messaging	GET conversations · POST conversations (enquiry: listing_id or supplier_id) · GET conversations/:id/messages · POST conversations/:id/messages · POST messages/read · GET realtime/token (Ably token request, capability-scoped)
misc	POST media/presign (kind-scoped: listing_photo/avatar/logo/verification_doc/handover_photo; content-type+size validated) · GET healthz · GET readyz · GET geocode?q= (LocationIQ proxy, server-side key, cached 24h)

Serialization rule (D-014): the Hire serializer is **role-shaped** — `service_fee` and `payout_amount` fields are present only when `request.user == hire.supplier` or `staff`. This is enforced in the serializer layer with tests, not left to clients.

3.9 Media pipeline

Client → `POST media/presign` (validates kind, content-type, ≤ size cap; returns key + presigned PUT) → direct R2 upload → attach key via the owning endpoint. Public bucket via R2 public domain (listing photos ≤10MB accepted but clients pre-resize to ≤1920px/~1MB via Expo ImageManipulator / portal canvas). Private bucket (verification docs, handover photos): presigned GET, 15-min expiry, owner+Ops only. Weekly orphan sweep deletes unattached keys >7 days old.

3.10 Masking implementation

On message create: regex Nigerian phones (`(\+?234|0)[789][01]\d{8}`) + spaced/dotted variants) + emails → `body_masked` stored alongside `body`. Serving: hire conversations serve `body` once the hire is paid, else `body_masked`; enquiry conversations always serve `body_masked`. Original `body` retained (Ops/dispute visibility). Masking notice is a UI concern (MaskedContact component, doc 08 §3.4).

4. Realtime (Aby, D-011)

- `GET /api/v1/realtime/token` → Aby TokenRequest, capabilities limited to the caller's channels.
- Channels: `conv:{conversation_id}` (new messages) · `user:{user_id}` (badge counts, hire status changes). Server publishes via Aby REST inside `transaction.on_commit`.
- Postgres rows are the message of record; Aby is fan-out only. Client degradation: 15s polling when Aby is unavailable (FSD §8). Free-tier budget: 200 concurrent / 6M msgs — alert at 70% via Aby stats in the weekly digest.

5. Supplier Portal

- Next.js 15 App Router via `@opennextjs/cloudflare` on Workers (D-012); Tailwind + shadcn/ui themed from `packages/tokens`; TanStack Query; react-hook-form + zod (mirrors DRF validation); `maplibre-gl` for yard/listing pins; Aby JS for live inbox.
- **BFF auth:** `/auth/*` route handlers proxy DRF; access+refresh JWTs in `httpOnly Secure SameSite=Lax` cookies; auto-refresh on 401 (single-flight); browser JS never sees tokens; all data calls via `/bff/*` adding Bearer.
- Routes per FSD §10.2. CalendarGantt = custom CSS grid (no heavyweight calendar dependency). DataTable, Stepper, LockedTerms, EventTimeline, StatusBadge per design-system chapters 05–06.

6. Hirer App

- Expo SDK (current) · TypeScript strict · expo-router · TanStack Query + Zustand (session, map filters) · tokens in SecureStore · NativeWind styled from `packages/tokens`.
- Map: `maplibre-react-native` + OpenFreeMap style URL (Protomaps-on-R2 fallback = config swap, D-013); pins/Yard Sheet per doc 08 §4.2; LocationIQ autocomplete via backend proxy.
- Payments: open `authorization_url` in expo-web-browser; on return **poll hire status** (webhook is truth). CountdownPill drives urgency.
- Handover: camera-first capture, client resize, presigned PUT. Offline posture: tile cache + persisted Query cache (MMKV) so My Hires/Messages render cold; mutations require connectivity.
- Distribution: local Android builds; EAS free tier iOS (15/mo) + expo-updates OTA for JS changes.

7. Security & Compliance Implementation

HTTPS/HSTS platform-level · bcrypt 12 · Fernet bank-number encryption (key rotation procedure documented in runbook) · private R2 presigned GET 15-min · Paystack HMAC + `paystack_events` dedup · CORS allowlist (portal

origin; app uses native fetch) · throttles §3.8 · Django Admin: staff-only, django-otp 2FA, separate cookie path, IP logging · audit = append-only `hire_events` + `LogEntry` retention · GDPR: registration consent copy, soft-delete + 30-day purge task with retention carve-outs (financial 7y, verification 5y) · secrets only in platform env · dependency updates: Dependabot weekly, security patches within 72h.

8. Testing Strategy

- **Backend:** pytest + factory-boy + freezegun + hypothesis. Mandatory suites: fee engine (FSD §3.1 vectors + property tests) · availability incl. **FOR UPDATE race test** (threaded double-payment on last unit) · state machine (every legal + illegal transition, guard failures) · webhook idempotency (duplicate event_id, out-of-order, replay) · sweep idempotency (double-run) · refund math per §7.6 row · D-014 serializer shaping (hirer never receives fee fields) · masking on/off conditions. Coverage gates: 85% `hires+payments`, 70% overall.
- **Portal:** vitest (BFF refresh logic, fee display logic) + 1 Playwright smoke (login → create listing → accept hire) against compose.
- **App:** TS strict; unit tests for price-preview + countdown hooks; manual E2E per release checklist (Detox deferred).
- **Pre-launch runbook:** the FSD §11.4 failure scenarios + 5 end-to-end Paystack-test-mode hires, documented and repeatable.

9. Observability & Operations

Sentry (4 surfaces, release-tagged) · structlog JSON → Railway logs · `/healthz` (app+DB) `/readyz` (DB, R2, Ably auth, Paystack ping) · reconciliation alert → Sentry + Ops email · weekly digest (GMV, fees, hires by state, Ably usage, R2 usage) → founder email · runbooks in `docs/runbooks/`: payout batch, refund handling, dispute resolution, key rotation, OpenFreeMap→Protomaps switch, Railway usage-cap response.

10. Build Waves (gated — founder approval required to start each wave)

Wave	Deliverable	Exit criterion
0	Monorepo, compose, Django skeleton (custom User, core, settings, healthz), tokens package, CI green, Railway+Workers deploy	<code>/healthz</code> live in prod; portal hello-world on Workers
1	Accounts: register/OTP/JWT/reset; VerificationRequest + Ops queue; account levels	Register→verify→login on prod API via <code>/api/docs/</code>
2	Supply: SupplierProfile, Yards, SpecTemplates seeded, Listings CRUD + photos + publish gates + duplicate + reports	Supplier takes a listing Live via API
3	Search: map endpoint (yard aggregation), list, filters incl. ★ specs	Map JSON correct for solo/yard/filtered/zero-match cases
4	Hires + money: state machine, availability+locks, fees, Paystack init/webhook/refunds, sweeps, payout queue, reconciliation	5 test hires E2E on Paystack test mode; race + idempotency tests green; zero reconciliation mismatches
5	Messaging: conversations, masking, Ably + polling fallback, unread	Two-device realtime chat; masking verified pre/post payment
6	Ops Console complete (queues, dashboard, dispute actions, digests)	Founder operates every queue without SQL
7	Supplier Portal complete (FSD §10.2)	Journey 1 (fleet onboarding) E2E in browser
8	Hirer app complete (FSD §10.1) — Android first, iOS via EAS	Journey 2+3 E2E on device

9	Hardening: map-path load test, security pass, Sentry triage, runbooks, beta onboarding	FSD \$13 launch gate green
---	--	----------------------------

Waves 7–8 may interleave with 5–6 once Wave 4's OpenAPI contracts are frozen.

11. Out-of-Scope Architecture Notes (Phase 2+ hooks already accommodated)

Paystack Transfers/split (payout model already shaped) · deposits (PB §7 tiers; card-hold flow) · `parent_hire` extensions · NIN/BVN/CAC API verification (`VerificationRequest.kind` extensible) · push (Expo notifications; device-token table added then) · reviews (post-completion hook on the state machine) · rate-config UI (`RATE_TABLE` → DB table) · multi-region/multi-currency (money stays integer minor-units).

— End of TSD v2.1 —